UNIT-3

Introduction to Statistical learning theory

Lecture-1

Statistical learning theory is a framework for machine learning that draws from statistics and functional analysis. It deals with finding a predictive function based on the data presented. The main idea in statistical learning theory is to build a model that can draw conclusions from data and make predictions.

Types of Data in Statistical Learning:

With statistical learning theory, there are two main types of data:

- Dependent Variable — a variable (y) whose values depend on the values of other variables (a dependent variable is sometimes also referred to as a target variable)

- Independent Variables — a variable (x) whose value does not depend on the values of other variables (independent variables are sometimes also referred to as predictor variables, input variables, explanatory variables, or features).

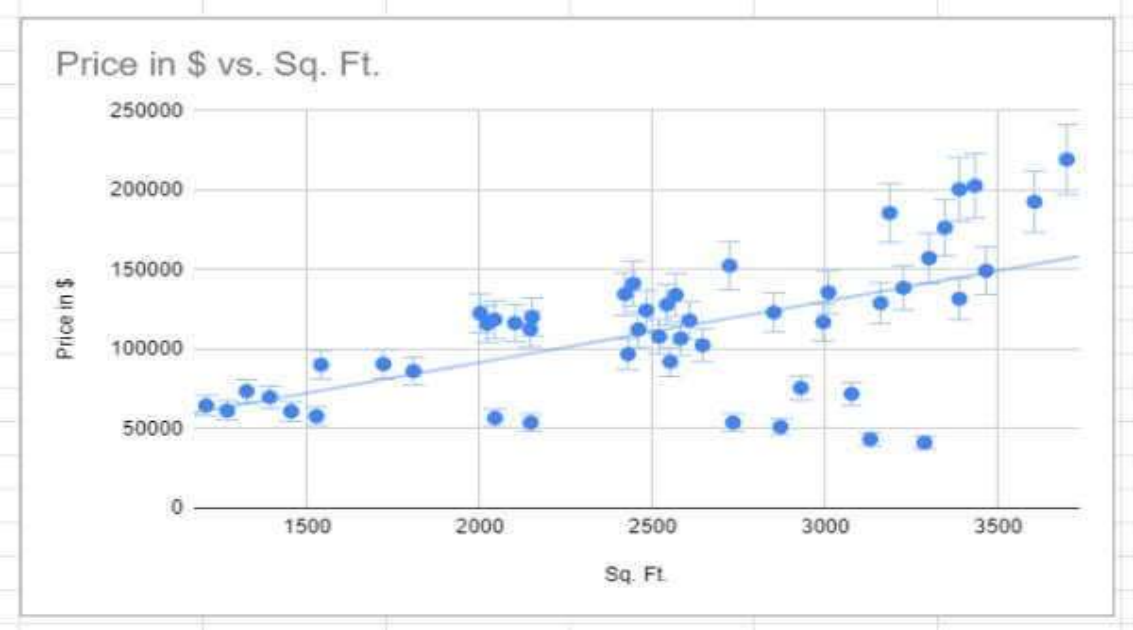
In statistical learning, the independent variable(s) are the variable that will affect the dependent variable. A common example of an Independent Variable is Age. There is nothing that one can do to increase or decrease age. This variable is independent.

Some common examples of Dependent Variables are:

- Weight — a person's weight is dependent on his or her age, diet, and activity levels (as well as other factors)

- Temperature — temperature is impacted by altitude, distance from equator (latitude) and distance from the sea

In graphs, the independent variable is often plotted along the x-axis while the dependent variable is plotted along the y-axis.



In this example, which shows how the price of a home is affected by the size of the home, sq. ft is the independent variable while price of the home is the dependent variable.

Statistical Model:

A statistical model defines the relationships between a dependent and independent variable. In the above graph, the relationships between the size of the home and the price of the home is illustrated by the straight line. We can define this relationship by using y = mx + c where m represents the gradient and c is the intercept. Another way that this equation can be expressed is with roman numerals which would look something like:

$$\text{Price of Home} = = \beta_0 + \beta_1 * \text{sq. ft}$$

This model would describe the price of a home as having a linear relationship with the size of a home. This would represent a simple model for the relationship.

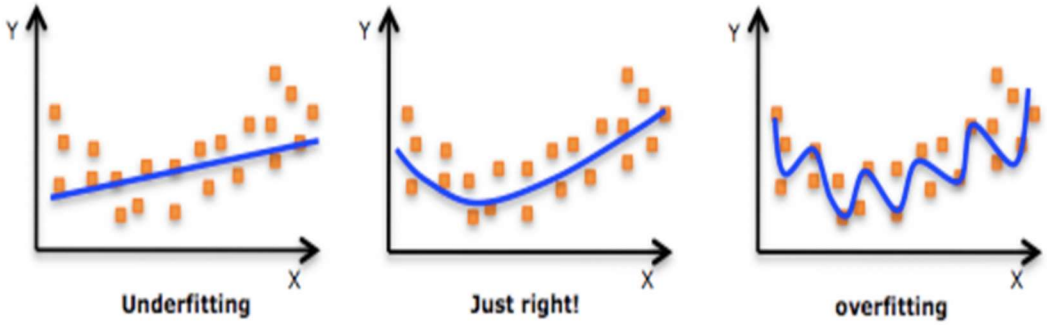
If we suppose that the size of the home is not the only independent variable when determining the price and that the number of bathrooms is also an independent variable, the equation would look like:

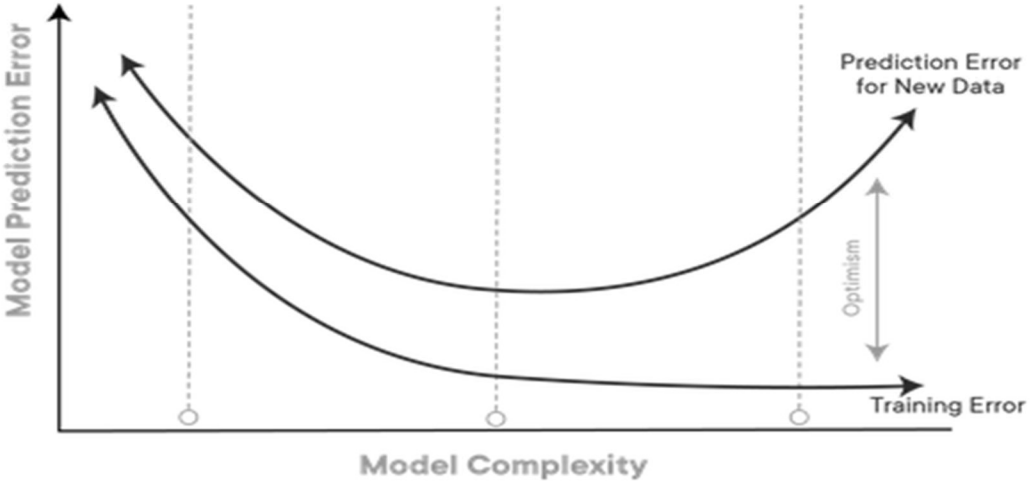
$$\text{Price of Home} = \beta_0 + \beta_1 * \text{sq. ft} + \beta_2 * \text{\# of bathrooms}$$

Model Generalization:

In order to build an effective model, the available data needs to be used in a way that would make the model generalizable for unseen situations. Some problems that occur when building models is that the model under-fits or over-fits to the data.

- Under-fitting — when a statistical model does not adequately capture the underlying structure of the data and, therefore, does not include some parameters that would appear in a correctly specified model.
- Over-fitting — when a statistical model contains more parameters that can be justified by the data and includes the residual variation ("noise") as if the variation represents underlying model structure**.**





As you can see, it is important to create a model that can generalize to the data that it is given so that it can make the most accurate predictions when given new data.

Model Validation:

Model Validation is used to assess over-fitting and under-fitting of the data. The steps to perform model validation are:

1. Split the data into two parts, training data and testing data (anywhere between 80/20 and 70/30 is ideal).

2. Use the larger portion (training data) to train the model.

3. Use the smaller portion (testing data) to test the model. This data is not being used to train the model, so it will be new data for the model to build predictions from.

4. If the model has learned well from the training data, it will perform well with both the training data and testing data. To determine how well the model is performing on both sets of data, you can calculate the accuracy score for each. The model is over-fitting if the training data has a significantly higher accuracy score than the testing data.

What is feature extraction?

Feature extraction is a process of dimensionality reduction by which an initial set of raw data is reduced to more manageable groups for processing. A characteristic of these large data sets is a large number of variables that require a lot of computing resources to process. Feature extraction is the name for methods that select and /or combine variables into features, effectively reducing the amount of data that must be processed, while still accurately and completely describing the original data set.

Feature extraction essentially is the process of converting raw data into numerical features that can be processed while preserving the information in the original data set. It yields better results than if you directly apply machine learning techniques on the raw data.

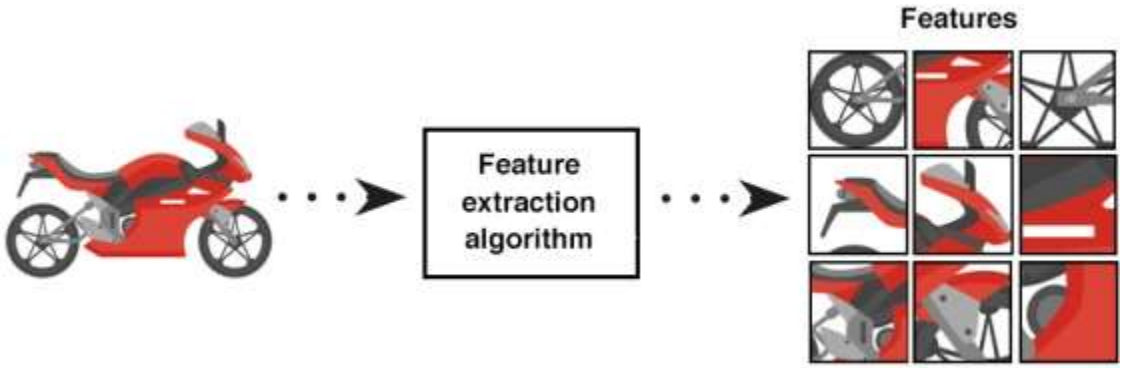
With the ascent of deep learning, feature extraction has been replaced to a great extent by the first layers of deep networks – but primarily for image data. For signal and time-series applications, feature extraction remains the first challenge that needs a significant amount expertise before you can get around to building effective predictive models.

How can feature extraction be carried out?

Feature extraction can be carried out either manually or automatically.

Manual feature extraction needs you to identify and describe the features that are relevant for a specific problem and implement a way to extract those features. In several situations, possessing a good understanding of the background or domain can help in making informed decisions regarding which features could turn out to be useful. Over decades of research, engineers and scientists have developed feature extraction methods for images, signals, and text. An example of a simple feature is the mean of a window in a signal.

Automated feature extraction involves making use of specialized algorithms or deep networks for the purpose of extracting features automatically from signals or images without the need for human intervention. This technique can be particularly useful when you wish to move quickly from raw data to developing machine learning algorithms. Wavelet scattering is an example of automated feature extraction.



Why is feature extraction used?

The process of feature extraction is useful when you need to reduce the number of resources needed for processing without losing important or relevant information. Feature extraction can also reduce the amount of redundant data for a given analysis. Also, the reduction of the data and the machine's efforts in building variable combinations (features) facilitate the speed of learning and generalization steps in the machine learning process, thus proving:

- Accuracy improvements
- Over fitting risk reduction
- Speed up in training
- Improved Data Visualization
- Increase in explain ability of our model

Feature Extraction aims to reduce the number of features in a dataset by creating new features from the existing ones (and then discarding the original features). These new reduced set of features should then be able to summarize most of the information contained in the original set of features. In this way, a summarized version of the original features can be created from a combination of the original set. Feature extraction reduces the amount of redundant data in the dataset, which helps build the model with less machine effort and even increases the speed of learning and generalization steps in the machine learning process.

What are the applications of feature extraction?



1. Auto-encoders

The purpose of auto encoders is unsupervised learning of efficient data coding. Feature extraction is used here to identify key features in the data for coding by learning from the coding of the original data set to derive new ones.

2. Bag-of-Words

Bag-of-words is a technique for natural language processing that extracts the words (features) used in a sentence, document, website, etc. and classifies them by frequency of use. This technique can also be applied to image processing.

3. Image Processing

Algorithms are used to detect features such as shaped, edges, or motion in a digital image or video.

How is feature extraction performed for image data?

Feature extraction for image data involves representing the interesting parts of an image in the form of a compact feature vector. In the past, this was carried out with specialized feature detection, feature extraction, and feature matching algorithms. Today, deep learning is prevalent in image and video analysis, and has become known for its ability to use raw image data as input, skipping the feature extraction step. Regardless of which approach you use, computer vision applications like image registration, object detection and classification, and content-based image retrieval, all need effective representation of image features. This could be either done implicitly by the first layers of a deep network, or explicitly applying some of the longstanding image feature extraction techniques.

What is the difference between feature selection and feature extraction?

Feature selection is used for the purpose of filtering irrelevant or redundant features from your dataset. The main difference between feature selection and extraction is that feature selection keeps a subset of the original features while feature extraction creates brand new ones.

How is feature extraction performed on signal features and time-frequency transformations?

Feature extraction will identify the most discriminating characteristics in signals, which a machine learning or a deep learning algorithm can more easily consume. Training machine learning or deep learning directly with raw signals often yields poor results because of the high data rate and information redundancy.

Apply pulse and transition metrics, measure signal-to-noise ratio (SNR), estimate spectral entropy and kurtosis, and compute power spectra when analyzing signals and sensor data.

Use time-frequency transformations, like the short-time Fourier transform (STFT) as signal representations for training data in machine learning and deep learning models. As an example, convolutional neural networks (CNNs) are often used on image data and can successfully learn from the 2D signal representations returned by time-frequency transformations.

Also use other time-frequency transformations, depending on the specific application or the characteristics. As an example, the constant-Q transform (CQT) offers a logarithmically spaced frequency distribution; the continuous wavelet transform (CWT) is usually effective at identifying short transients in non-stationary signals.

Lecture-2

Principal Component Analysis

Principal Component Analysis is an unsupervised learning algorithm that is used for the dimensionality reduction in machine learning. It is a statistical process that converts the observations of correlated features into a set of linearly uncorrelated features with the help of orthogonal transformation. These new transformed features are called the **Principal Components**. It is one of the popular tools that is used for exploratory data analysis and predictive modeling. It is a technique to draw strong patterns from the given dataset by reducing the variances.

PCA generally tries to find the lower-dimensional surface to project the high-dimensional data.

PCA works by considering the variance of each attribute because the high attribute shows the good split between the classes, and hence it reduces the dimensionality. Some real-world applications of PCA are ***image processing, movie recommendation system, optimizing the power allocation in various communication channels.*** It is a feature extraction technique, so it contains the important variables and drops the least important variable.

The PCA algorithm is based on some mathematical concepts such as:

- Variance and Covariance
- Eigenvalues and Eigen factors

Some common terms used in PCA algorithm:

- **Dimensionality:** It is the number of features or variables present in the given dataset. More easily, it is the number of columns present in the dataset.
- **Correlation:** It signifies that how strongly two variables are related to each other. Such as if one changes, the other variable also gets changed. The correlation value ranges from -1 to +1. Here, -1 occurs if variables are inversely proportional to each other, and +1 indicates that variables are directly proportional to each other.
- **Orthogonal:** It defines that variables are not correlated to each other, and hence the correlation between the pair of variables is zero.
- **Eigenvectors:** If there is a square matrix M, and a non-zero vector v is given. Then v will be eigenvector if Av is the scalar multiple of v.
- **Covariance Matrix:** A matrix containing the covariance between the pair of variables is called the Covariance Matrix.

Principal Components in PCA

As described above, the transformed new features or the output of PCA are the Principal Components. The number of these PCs is either equal to or less than the original features present in the dataset. Some properties of these principal components are given below:

- The principal component must be the linear combination of the original features.
- These components are orthogonal, i.e., the correlation between a pair of variables is zero.
- The importance of each component decreases when going to 1 to n, it means the 1 PC has the most importance, and n PC will have the least importance.

Steps for PCA algorithm

1. Getting the dataset
Firstly, we need to take the input dataset and divide it into two subparts X and Y, where X is the training set, and Y is the validation set.

2. Representing data into a structure

Represent our dataset into a structure. Such as we will represent the two-dimensional matrix of independent variable X. Here each row corresponds to the data items, and the column corresponds to the Features. The number of columns is the dimensions of the dataset.

3. Standardizing the data

In a particular column, the features with high variance are more important compared to the features with lower variance.

If the importance of features is independent of the variance of the feature, then we will divide each data item in a column with the standard deviation of the column. Here we will name the matrix as Z.

4. **Calculating the Covariance of Z**

To calculate the covariance of Z, we will take the matrix Z, and will transpose it. After transpose, we will multiply it by Z. The output matrix will be the Covariance matrix of Z.

5. **Calculating the Eigen Values and Eigen Vectors**

To calculate the eigenvalues and eigenvectors for the resultant covariance matrix Z. Eigenvectors or the covariance matrix are the directions of the axes with high information. And the coefficients of these eigenvectors are defined as the eigenvalues.

6. **Sorting the Eigen Vectors**

In this step, we will take all the eigenvalues and will sort them in decreasing order, which means from largest to smallest. And simultaneously sort the eigenvectors accordingly in matrix P of eigenvalues. The resultant matrix will be named as P*.

7. **Calculating the new features Or Principal Components**

Here we will calculate the new features. To do this, we will multiply the P* matrix to the Z. In the resultant matrix Z*, each observation is the linear combination of original features. Each column of the Z* matrix is independent of each other.

8. Remove less or unimportant features from the new dataset.

The new feature set has occurred, so we will decide here what to keep and what to remove. It means, we will only keep the relevant or important features in the new dataset, and unimportant features will be removed out.

Applications of Principal Component Analysis

- PCA is mainly used as the dimensionality reduction technique in various AI applications such **as computer vision, image compression, etc.**
- It can also be used for finding hidden patterns if data has high dimensions. Some fields where PCA is used are Finance, data mining, Psychology, etc.

Lecture-3

Singular Value Decomposition

- Singular Value Decomposition is one of the important concepts in linear algebra. To understand the meaning of singular value decomposition (SVD), one must be aware of the related concepts such as matrix, types of matrices, transformations of a matrix, etc. As this concept is connected to various concepts of linear algebra, it's become challenging to learn the singular value decomposition of a matrix. In this article, you will learn the definition of singular value decomposition, examples of 2×2 and 3×3 matrix decomposition in detail.

What is Singular Value Decomposition?

- The Singular Value Decomposition of a matrix is a factorization of the matrix into three matrices. Thus, the singular value decomposition of matrix A can be expressed in terms of the factorization of A into the product of three matrices as $A = UDV^T$
- Here, the columns of U and V are orthonormal, and the matrix D is diagonal with real positive entries.

Singular Value Decomposition of a Matrix

Mathematically, the singular value decomposition of a matrix can be explained as follows:

Consider a matrix A of order mxn.

This can be uniquely decomposed as:

$A = UDV^T$

U is mxn and column orthogonal (that means its columns are eigenvectors of AA^T)

($AA^T = UDV^T VDU^T = UD^2U^T$)

V is nxn and orthogonal (that means its columns are eigenvectors of A^TA)

($A^TA = VDU^T UDV^T = VD^2V^T$)

D is nxn diagonal, where non-negative real values are called singular values.

Learn how to find eigenvalues and eigenvectors of a matrix here.

Let $D = diag(\sigma_1, \sigma_2, \ldots, \sigma_n)$ ordered such that $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_n$.

If σ is a singular value of A, its square is an eigenvalue of A^TA.

Also, let $U = (u_1\ u_2\ \ldots\ u_n)$ and $V = (v_1\ v_2\ \ldots\ v_n)$.

Therefore,

$A = \sum_{i=1}^{n} \sigma_i u_i v_i^T$

Here, the sum can be given from 1 to r so that r is the rank of matrix A.

Go through the example given below to understand the process of singular value decomposition of a matrix in a better way.

Lecture-4

Singular Value Decomposition Example

The process of finding the singular value decomposition for 3×3 matrix and 2×2 matrix is the same. Example of 2×2 matrix decomposition,

Find the singular value decomposition of a matrix

$$A = \begin{bmatrix} -4 & -7 \\ 1 & 4 \end{bmatrix}$$

.

Solution:

Given,

$$A = \begin{bmatrix} -4 & -7 \\ 1 & 4 \end{bmatrix}$$

So,

$$A^T = \begin{bmatrix} -4 & 1 \\ -7 & 4 \end{bmatrix}$$

Now,

$$AA^T = \begin{bmatrix} -4 & -7 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} -4 & 1 \\ -7 & 4 \end{bmatrix} = \begin{bmatrix} 65 & -32 \\ -32 & 17 \end{bmatrix}$$

Finding the eigenvector for AAT.

$$|A \cdot A' - \lambda I| = 0$$

$$\begin{vmatrix} (65 - \lambda) & -32 \\ -32 & (17 - \lambda) \end{vmatrix} = 0$$

$$\therefore (65 - \lambda) \times (17 - \lambda) - (-32) \times (-32) = 0$$

$$\therefore \left(1105 - 82\lambda + \lambda^2\right) - 1024 = 0$$

$$\therefore \left(\lambda^2 - 82\lambda + 81\right) = 0$$

$$\therefore (\lambda - 1)(\lambda - 81) = 0$$

$$\therefore (\lambda - 1) = 0 \text{ or } (\lambda - 81) = 0$$

∴ The eigenvalues of the matrix A·A' are given by λ = 1, 81.

Now,

Eigenvectors for λ = 81 are:

$$v_1 = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$

Eigenvectors for λ = 1 are:

$$v_2 = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix}$$

For Eigenvector-1 $(-2, 1)$, Length L = $\sqrt{(-2)^2 + 1^2} = 2.236$

So, normalizing gives $u_1 = \left(\frac{-2}{2.236}, \frac{1}{2.236}\right) = (-0.894, 0.447)$

For Eigenvector-2 $(0.5, 1)$, Length L = $\sqrt{0.5^2 + 1^2} = 1.118$

So, normalizing gives $u_2 = \left(\frac{0.5}{1.118}, \frac{1}{1.118}\right) = (0.447, 0.894)$

Similarly, we can find the eigenvectors for A'A as:

Eigenvectors for λ = 81 are:

$v_1 = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix}$

Eigenvectors for λ = 1 are:

$v_2 = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$

For Eigenvector-1 $(0.5, 1)$, Length L = $\sqrt{0.5^2 + 1^2} = 1.118$

So, normalizing gives $v_1 = \left(\frac{0.5}{1.118}, \frac{1}{1.118}\right) = (0.447, 0.894)$

For Eigenvector-2 $(-2, 1)$, Length L = $\sqrt{(-2)^2 + 1^2} = 2.236$

So, normalizing gives $v_2 = \left(\frac{-2}{2.236}, \frac{1}{2.236} \right) = (- 0.894, 0.447)$

Using these values, we can write the solution.

$$\therefore \Sigma = \begin{bmatrix} \sqrt{81} & 0 \\ 0 & \sqrt{1} \end{bmatrix} = \begin{bmatrix} 9 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\therefore U = \left[u_1, u_2\right] = \begin{bmatrix} -0.894 & 0.447 \\ 0.447 & 0.894 \end{bmatrix}$$

V is found using formula $v_i = \frac{1}{\sigma_i} A^T \cdot u_i$

$$\therefore V = \begin{bmatrix} 0.447 & -0.894 \\ 0.894 & 0.447 \end{bmatrix}$$

U is found using formula $u_i = \frac{1}{\sigma_i} A \cdot v_i$

$$\therefore U = \begin{bmatrix} -0.894 & 0.447 \\ 0.447 & 0.894 \end{bmatrix}$$

Singular Value Decomposition Applications

Some of the applications of singular value decomposition are listed below:

- SVD has some fascinating algebraic characteristics and conveys relevant geometrical and theoretical insights regarding linear transformations.
- SVD has some critical applications in data science too.
- Mathematical applications of the SVD involve calculating the matrix approximation, rank of a matrix and so on.
- The SVD is also greatly useful in science and engineering.
- It has some applications of statistics, for example, least-squares fitting of data and process control.

Lecture-5

Feature Selection Techniques in Machine Learning

What is Feature Selection?

A feature is an attribute that has an impact on a problem or is useful for the problem, and choosing the important features for the model is known as feature selection. Each machine learning process depends on feature engineering, which mainly contains two processes; which are Feature Selection and Feature Extraction. Although feature selection and extraction processes may have the same objective, both are completely different from each other. The main difference between them is that feature selection is about selecting the subset of the original feature set, whereas feature extraction creates new features. Feature selection is a way of reducing the input variable for the model by using only relevant data in order to reduce overfitting in the model.

So, we can define feature Selection as, "***It is a process of automatically or manually selecting the subset of most appropriate and relevant features to be used in model building***." Feature selection is performed by either including the important features or excluding the irrelevant features in the dataset without changing them.

Need for Feature Selection

Before implementing any technique, it is really important to understand, need for the technique and so for the Feature Selection. As we know, in machine learning, it is necessary to provide a pre-processed and good input dataset in order to get better outcomes. We collect a huge amount of data to train our model and help it to learn better. Generally, the dataset consists of noisy data, irrelevant data, and some part of useful data. Moreover, the huge amount of data also slows down the training process of the model, and with noise and irrelevant data, the model may not predict and perform well. So, it is very necessary to remove such noises and less-important data from the dataset and to do this, and Feature selection techniques are used.

Selecting the best features helps the model to perform well. For example, Suppose we want to create a model that automatically decides which car should be crushed for a spare part, and to do this, we have a dataset. This dataset contains a Model of the car, Year, Owner's name, Miles. So, in this dataset, the name of the owner does not contribute to the model performance as it does not decide if the car should be crushed or not, so we can remove this column and select the rest of the features(column) for the model building.

Below are some benefits of using feature selection in machine learning:

- **It helps in avoiding the curse of dimensionality.**
- **It helps in the simplification of the model so that it can be easily interpreted by the researchers.**

- **It reduces the training time.**
- **It reduces overfitting hence enhance the generalization.**

Feature Selection Techniques

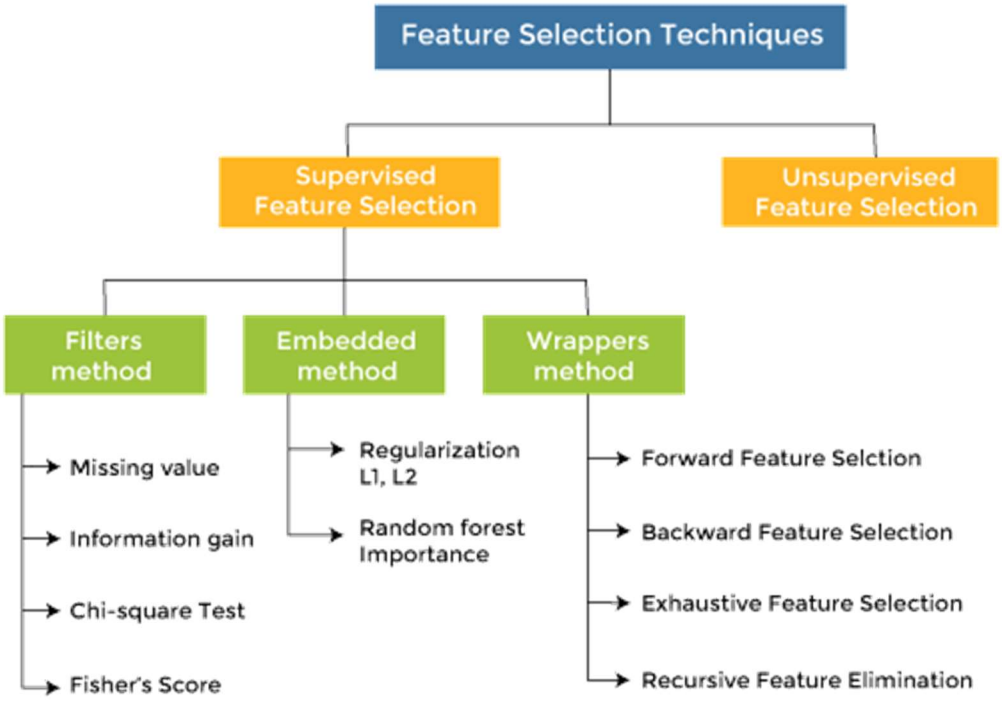
There are mainly two types of Feature Selection techniques, which are:

- **Supervised Feature Selection technique**

  Supervised Feature selection techniques consider the target variable and can be used for the labelled dataset.

- **UnSupervised Feature Selection technique**

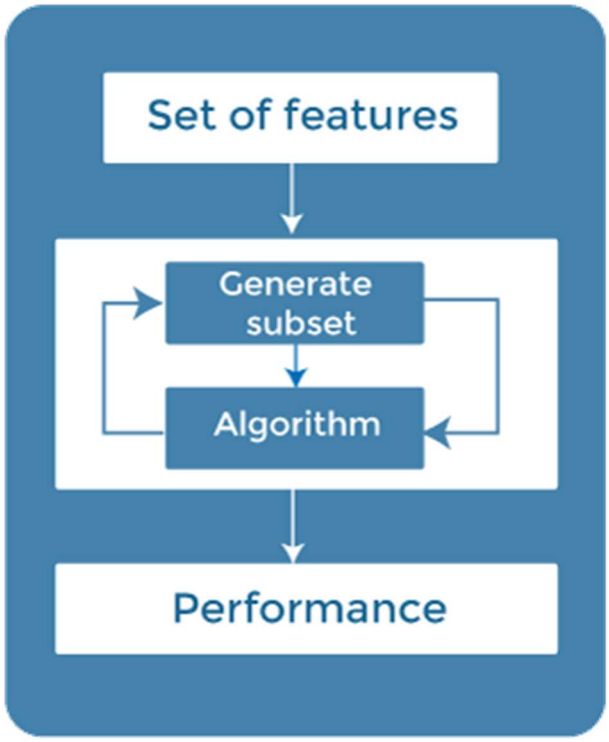
  Unsupervised Feature selection techniques ignore the target variable and can be used for the unlabelled dataset.



There are mainly three techniques under supervised feature Selection:

1. Wrapper Methods

In wrapper methodology, selection of features is done by considering it as a search problem, in which different combinations are made, evaluated, and compared with other combinations. It trains the algorithm by using the subset of features iteratively.



On the basis of the output of the model, features are added or subtracted, and with this feature set, the model has trained again.

Some techniques of wrapper methods are:

- **Forward selection** - Forward selection is an iterative process, which begins with an empty set of features. After each iteration, it keeps adding on a feature and evaluates the performance to check whether it is improving the performance or not. The process continues until the addition of a new variable/feature does not improve the performance of the model.
- **Backward elimination** - Backward elimination is also an iterative approach, but it is the opposite of forward selection. This technique begins the process by considering all the features and removes the least significant feature. This elimination process continues until removing the features does not improve the performance of the model.

- **Exhaustive Feature Selection-** Exhaustive feature selection is one of the best feature selection methods, which evaluates each feature set as brute-force. It means this method tries & make each possible combination of features and return the best performing feature set.
- **Recursive Feature Elimination-**

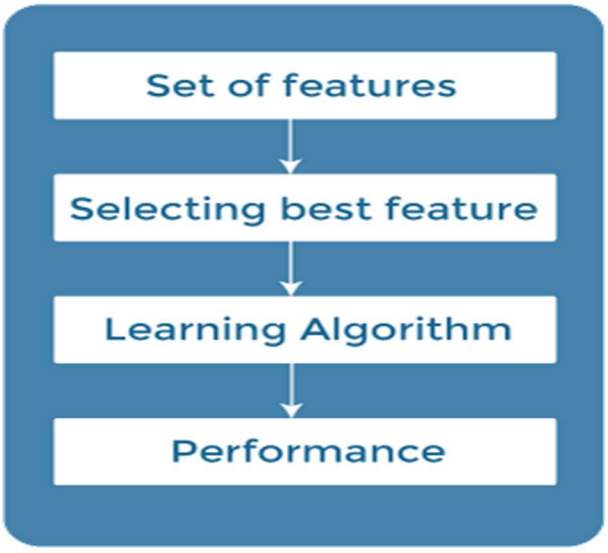
  Recursive feature elimination is a recursive greedy optimization approach, where features are selected by recursively taking a smaller and smaller subset of features. Now, an estimator is trained with each set of features, and the importance of each feature is determined using *coef_attribute* or through a *feature_importances_attribute.*

2. Filter Methods

In Filter Method, features are selected on the basis of statistics measures. This method does not depend on the learning algorithm and chooses the features as a pre-processing step.

The filter method filters out the irrelevant feature and redundant columns from the model by using different metrics through ranking.

The advantage of using filter methods is that it needs low computational time and does not overfit the data.



Some common techniques of Filter methods are as follows:

- Information Gain

- Chi-square Test
- Fisher's Score
- Missing Value Ratio

Information Gain: Information gain determines the reduction in entropy while transforming the dataset. It can be used as a feature selection technique by calculating the information gain of each variable with respect to the target variable.

Chi-square Test: Chi-square test is a technique to determine the relationship between the categorical variables. The chi-square value is calculated between each feature and the target variable, and the desired number of features with the best chi-square value is selected.

Fisher's Score:

Fisher's score is one of the popular supervised technique of features selection. It returns the rank of the variable on the fisher's criteria in descending order. Then we can select the variables with a large fisher's score.

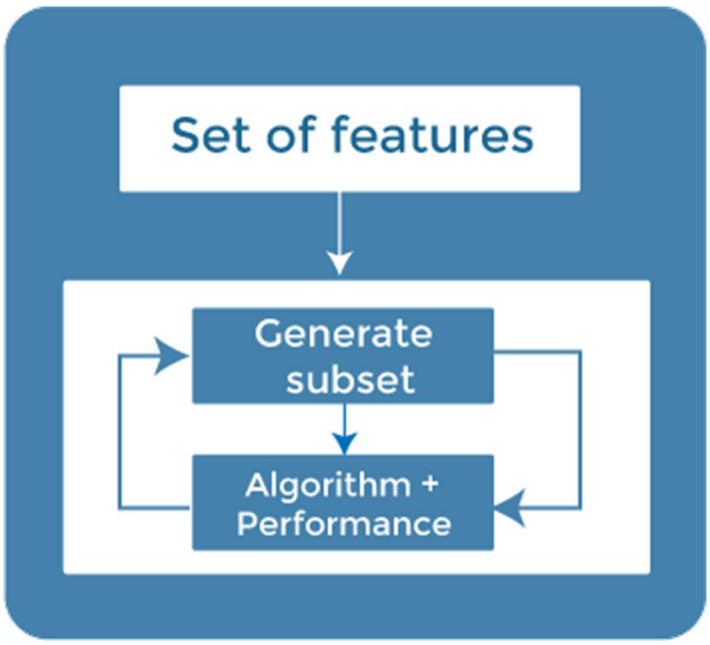
Missing Value Ratio:

The value of the missing value ratio can be used for evaluating the feature set against the threshold value. The formula for obtaining the missing value ratio is the number of missing values in each column divided by the total number of observations. The variable is having more than the threshold value can be dropped.

$$\text{Missing Value Ratio} = \frac{Number\ of\ Missing\ values * 100}{Total\ number\ of\ observations}$$

3. Embedded Methods

Embedded methods combined the advantages of both filter and wrapper methods by considering the interaction of features along with low computational cost. These are fast processing methods similar to the filter method but more accurate than the filter method.



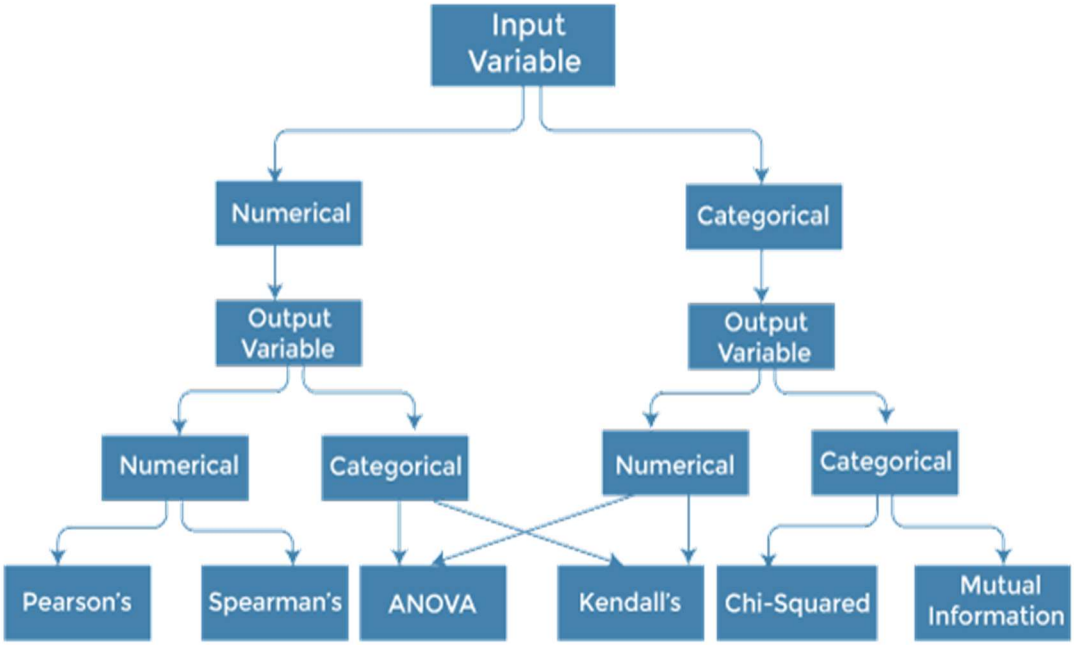
These methods are also iterative, which evaluates each iteration, and optimally finds the most important features that contribute the most to training in a particular iteration. Some techniques of embedded methods are:

- **Regularization**- Regularization adds a penalty term to different parameters of the machine learning model for avoiding overfitting in the model. This penalty term is added to the coefficients; hence it shrinks some coefficients to zero. Those features with zero coefficients can be removed from the dataset. The types of regularization techniques are L1 Regularization (Lasso Regularization) or Elastic Nets (L1 and L2 regularization).
- **Random Forest Importance** - Different tree-based methods of feature selection help us with feature importance to provide a way of selecting features. Here, feature importance specifies which feature has more importance in model building or has a great impact on the target variable. Random Forest is such a tree-based method, which is a type of bagging algorithm that aggregates a different number of decision trees. It automatically ranks the nodes by their performance or decrease in the impurity (Gini impurity) over all the trees. Nodes are arranged as per the impurity values, and thus it allows to pruning of trees below a specific node. The remaining nodes create a subset of the most important features.

Lecture-6

How to choose a Feature Selection Method?

For machine learning engineers, it is very important to understand that which feature selection method will work properly for their model. The more we know the datatypes of variables, the easier it is to choose the appropriate statistical measure for feature selection.



To know this, we need to first identify the type of input and output variables. In machine learning, variables are of mainly two types:

- **Numerical Variables:** Variable with continuous values such as integer, float
- **Categorical Variables:** Variables with categorical values such as Boolean, ordinal, nominals.

Below are some univariate statistical measures, which can be used for filter-based feature selection:

1. Numerical Input, Numerical Output:

Numerical Input variables are used for predictive regression modelling. The common method to be used for such a case is the Correlation coefficient.

- Pearson's correlation coefficient (For linear Correlation).

- Spearman's rank coefficient (for non-linear correlation).

2. Numerical Input, Categorical Output:

Numerical Input with categorical output is the case for classification predictive modelling problem**s.** In this case, also, correlation-based techniques should be used, but with categorical output.

- **ANOVA correlation coefficient (linear).**
- **Kendall's rank coefficient (nonlinear).**

3. Categorical Input, Numerical Output:

This is the case of regression predictive modelling with categorical input. It is a different example of a regression problem. We can use the same measures as discussed in the above case but in reverse order.

4. Categorical Input, Categorical Output:

This is a case of classification predictive modelling with categorical Input variables.

The commonly used technique for such a case is Chi-Squared Test. We can also use Information gain in this case.

We can summarise the above cases with appropriate measures in the below table:

Input Variable	Output Variable	Feature Selection technique
Numerical	Numerical	○ Pearson's correlation coefficient (For linear Correlation). ○ Spearman's rank coefficient (for non-linear correlation).
Numerical	Categorical	○ ANOVA correlation coefficient (linear). ○ Kendall's rank coefficient (nonlinear).
Categorical	Numerical	○ Kendall's rank coefficient (linear).

		○ ANOVA correlation coefficient (nonlinear).
Categorical	Categorical	○ Chi-Squared test (contingency tables). ○ Mutual Information.

Evaluating Machine Learning algorithms and Model Selection

What is model evaluation?

Model evaluation is a process of assessing the model's performance on a chosen evaluation setup. It is done by calculating quantitative performance metrics like F1 score or RMSE or assessing the results qualitatively by the subject matter experts. The machine learning **evaluation metrics you choose should reflect the business metrics** you want to optimize with the machine learning solution.

What is model selection?

Model selection is the process of choosing the best ml model for a given task. It is done by comparing various model candidates on chosen evaluation metrics calculated on a designed evaluation schema. Choosing the **correct evaluation schema**, whether a simple train test split or a complex cross-validation strategy, is the **crucial first step** of building any machine learning solution.

How to evaluate machine learning models and select the best one?

Step 1: Choose a proper validation strategy. Can't stress this enough, without a reliable way to validate your model performance, no amount of hyperparameter tuning and state-of-the-art models will help you.

Step 2: Choose the right evaluation metric. Figure out the business case behind your model and try to use the machine learning metric that correlates with that. Typically no one metric is ideal for the problem.
So calculate multiple metrics and make your decisions based on that. Sometimes you need to combine classic ML metrics with a subject matter expert evaluation. And that is ok.

Step 3: Keep track of your experiment results. Whether you use a spreadsheet or a dedicated experiment tracker, make sure to log all the important metrics, learning curves, dataset versions, and configurations.

Step 4: Compare experiments and pick a winner. Regardless of the metrics and validation strategy you choose, at the end of the day, you want to find the best model. But no model is really best, but some are good enough.

So make sure to understand what is good enough for your problem, and once you hit that, move on to other parts of the project, like model deployment or pipeline orchestration.

Lecture-7

Model selection in machine learning (choosing model validation strategy)

Resampling methods

Resampling methods, as the name suggests, are simple techniques of rearranging data samples to inspect if the model performs well on data samples that it has not been trained on. In other words, resampling helps us understand **if the model will generalize well**.

Random Split

Random Splits are used to randomly sample a percentage of data into training, testing, and preferably validation sets. The advantage of this method is that there is a good chance that the original population is well represented in all the three sets. In more formal terms, random splitting will prevent a biased sampling of data.

It is very important to note the use of the validation set in model selection. The validation set is the second test set and one might ask, why have two test sets?

In the process of feature selection and model tuning, the test set is used for model evaluation. This means that the model parameters and the feature set are selected such that they give an optimal result on the test set. Thus, the validation set which has completely unseen data points (not been used in the tuning and feature selection modules) is used for the final evaluation.

Time-Based Split

There are **some types of data where random splits are not possible**. For example, if we have to train a model for weather forecasting, we cannot randomly divide the data into training and testing sets. This will jumble up the seasonal pattern! Such data is often referred to by the term – Time Series.
In such cases, a time-wise split is used. The training set can have data for the last three years and 10 months of the present year. The last two months can be reserved for the testing or validation set.
There is also a **concept of *window* sets** – where the model is trained till a particular date and tested on the future dates iteratively such that the training window keeps increasing shifting by one day (consequently, the test set also reduces by a day). The advantage of this method is that it stabilizes the model and prevents overfitting when the test set is very small (say, 3 to 7 days).

However, the drawback of time-series data is that the events or data points are not ***mutually independent***. One event might affect every data input that follows after.
For instance, a change in the governing party might considerably change the population statistics for the years to follow. Or the infamous coronavirus pandemic is going to have a massive impact on economic data for the next few years.

No machine learning model can learn from past data in such a case because the data points before and after the event have major differences.

K-Fold Cross-Validation

The cross-validation technique works by **randomly shuffling the dataset and then splitting it into k groups.** Thereafter, on iterating over each group, the group needs to be considered as a test set while all other groups are clubbed together into the training set. The model is tested on the test group and the process continues for k groups.
Thus, by the end of the process, one has k different results on k different test groups. The best model can then be selected easily by choosing the one with the highest score.

Bootstrap

Bootstrap is one of the most powerful ways to obtain a stabilized model. It is close to the random splitting technique since it follows the concept of random sampling.
The first step is to select a sample size (which is usually equal to the size of the original dataset). Thereafter, a sample data point must be randomly selected from the original dataset and added to the bootstrap sample. After the addition, the sample needs to be put back into the original sample. This process needs to be repeated for N times, where N is the sample size.

Therefore, it is a resampling technique that creates the bootstrap sample by sampling data points from the original dataset ***with replacement***. This means that the bootstrap sample can contain multiple instances of the same data point.

The model is trained on the bootstrap sample and then evaluated on all those data points that did not make it to the bootstrapped sample. These are called the ***out-of-bag*** samples.

Akaike Information Criterion (AIC)

It is common knowledge that every model is not completely accurate. There is always some information loss which can be measured using the KL information metric. Kulback-Liebler or KL divergence is the measure of the difference in the probability distribution of two variables.
A statistician, Hirotugu Akaike, took into consideration the relationship between KL Information and Maximum Likelihood (in maximum-likelihood, one wishes to maximize the conditional probability of observing a datapoint X, given the parameters and a specified probability distribution) and developed the concept of Information Criterion (or IC). Therefore, Akaike's IC or AIC is the measure of information loss. This is how the discrepancy between two different models is captured and the model with the least information loss is suggested as the model of choice.

$$AIC = (2K - 2log(L))/N$$

K = number of independent variables or predictors
L = maximum-likelihood of the model
N = number of data points in the training set (especially helpful in case of small datasets)

The limitation of AIC is that it is not very good with generalizing models as it tends to select complex models that lose less training information.

Bayesian Information Criterion (BIC)

BIC was derived from the Bayesian probability concept and is suited for models that are trained under the maximum likelihood estimation.

$$BIC = K * log(N) - 2log(L)$$

K = number of independent variables
L = maximum-likelihood
N = Number of sampler/data points in the training set.

BIC penalizes the model for its complexity and is preferably used when the size of the dataset is not very small (otherwise it tends to settle on very simple models).

Lecture-8

How to evaluate ML models (choosing performance metrics)

Models can be evaluated using multiple metrics. However, the right choice of an evaluation metric is crucial and often depends upon the problem that is being solved. A clear understanding of a wide range of metrics can help the evaluator to chance upon an appropriate match of the problem statement and a metric.

i) Classification metrics:

For every classification model prediction, a matrix called the *confusion matrix* can be constructed which demonstrates the number of test cases correctly and incorrectly classified.
It looks something like this (considering 1 -Positive and 0 -Negative are the target classes):

TN: Number of negative cases correctly classified
TP: Number of positive cases correctly classified
FN: Number of positive cases incorrectly classified as negative
FP: Number of negative cases correctly classified as positive

Accuracy

Accuracy is the simplest metric and can be defined as the number of test cases correctly classified divided by the total number of test cases.

$$Accuracy = (TP+TN)/(TP+TN+FP+FN)$$

It can be applied to most generic problems but is not very useful when it comes to unbalanced datasets.
For instance, if we are detecting frauds in bank data, the ratio of fraud to non-fraud cases can be 1:99. In such cases, if accuracy is used, the model will turn out to be 99% accurate by predicting all test cases as non-fraud. The 99% accurate model will be completely useless.

If a model is poorly trained such that it predicts all the 1000 (say) data points as non-frauds, it will be missing out on the 10 fraud data points. If accuracy is measured, it will show that that model correctly predicts 990 data points and thus, it will have an accuracy of (990/1000)*100 = 99%!
This is why accuracy is a false indicator of the model's health.

Therefore, for such a case, a metric is required that can focus on the ten fraud data points which were completely missed by the model.

Precision

Precision is the metric used to identify the correctness of classification.

$$Precision = TP/(TP + FP)$$

Intuitively, this equation is the ratio of correct positive classifications to the total number of predicted positive classifications. The greater the fraction, the higher is the precision, which means better is the ability of the model to correctly classify the positive class.

In the problem of predictive maintenance (where one must predict in advance when a machine needs to be repaired), precision comes into play. The cost of maintenance is usually high and thus, incorrect predictions can lead to a loss for the company. In such cases, the ability of the model to correctly classify the positive class and to lower the number of false positives is paramount.

Recall

Recall tells us the number of positive cases correctly identified out of the total number of positive cases.

$$Recall = TP/(TP + FN)$$

Going back to the fraud problem, the recall value will be very useful in fraud cases because a high recall value will indicate that a lot of fraud cases were identified out of the total number of frauds.

F1 Score

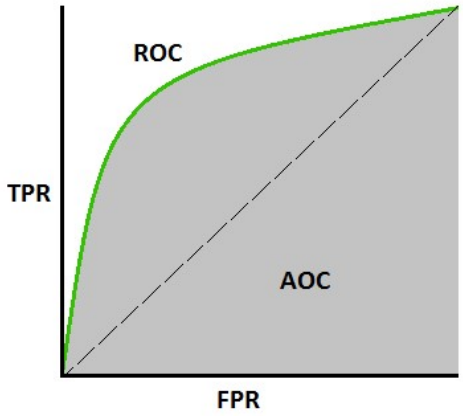
F1 score is the harmonic mean of Recall and Precision and therefore, balances out the strengths of each.
It is useful in cases where both recall and precision can be valuable – like in the identification of plane parts that might require repairing. Here, precision will be required to save on the company's cost (because plane parts are extremely expensive) and recall will be required to ensure that the machinery is stable and not a threat to human lives.

$$F1Score = 2 * ((precision * recall)/(precision + recall))$$

AUC-ROC

ROC curve is a plot of **true positive rate** (recall) against **false positive rate** (TN / (TN+FP)). AUC-ROC stands for Area under the Receiver Operating Characteristics and the higher the area, the better is the model performance.

If the curve is somewhere near the 50% diagonal line, it suggests that the model randomly predicts the output variable.



AUC - ROC Curve [Image 2] (Image courtesy: My Photoshopped Collection)

Gain and Lift Charts

Gain and lift charts are tools that evaluate model performance just like the confusion matrix but with a subtle, yet significant difference. The confusion matrix determines the performance of the model on the whole population or the entire test set, whereas the gain and lift charts evaluate the model on portions of the whole population. Therefore, we have a score (y-axis) for every % of the population (x-axis).

Lift charts measure the improvement that a model brings in compared to random predictions. The improvement is referred to as the 'lift'.

Regression metrics

Regression models provide a continuous output variable, unlike classification models that have discrete output variables. Therefore, the metrics for assessing the regression models are accordingly designed.

Mean Squared Error or MSE

MSE is a simple metric that calculates the difference between the actual value and the predicted value (error), squares it and then provides the mean of all the errors.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \widetilde{y_i})^2$$

MSE is very sensitive to outliers and will show a very high error value even if a few outliers are present in the otherwise well-fitted model predictions.

Root Mean Squared Error or RMSE

RMSE is the root of MSE and is beneficial because it helps to bring down the scale of the errors closer to the actual values, making it more interpretable.

Mean Absolute Error or MAE

MAE is the mean of the absolute error values (actuals – predictions).

$$MAE = \frac{1}{n}\sum_{i=1} |x_i - x|$$

If one wants to ignore the outlier values to a certain degree, MAE is the choice since it reduces the penalty of the outliers significantly with the removal of the square terms.

Root Mean Squared Log Error or RMSLE

In RMSLE, the same equation as that of RMSE is followed except for an added log function along with the actual and predicted values.

$$RMSLE = \sqrt{\frac{1}{n}\Sigma_{i=1}^{n}(log(x_i + 1)) - (log(y_i + 1))^2}$$

x is the actual value and y is the predicted value. This helps to scale down the effect of the outliers by downplaying the higher error rates with the log function. Also, RMSLE helps to capture a **relative error** (by comparing all the error values) through the use of logs.

R-Squared

R-Square helps to identify the proportion of variance of the target variable that can be captured with the help of the independent variables or predictors.

$$R\text{-}Squared = 1-(UnexplainedVariation/TotalVariation) = 1-(Variance(model)/Variance(Average))$$

R-square, however, has a gigantic problem. Say, a new unrelated feature is added to a model with an assigned weight of w. If the model finds absolutely no correlation between the new predictor and the target variable, w is 0. However, there is almost always a small correlation due to randomness which adds a small positive weight (w>0) and a new loss minimum is achieved due to overfitting.

This is why the R-squared increases with any new feature addition. Thus, its **inability to decrease in value when new features are added** limits its ability to identify if the model did better with lesser features.

ii)Clustering metrics:

Clustering algorithms predict groups of datapoints and hence, distance-based metrics are most effective.

Dunn Index

Dunn Index focuses on identifying clusters that have low variance (among all members in the cluster) and are compact. The mean values of the different clusters also need to be far apart.

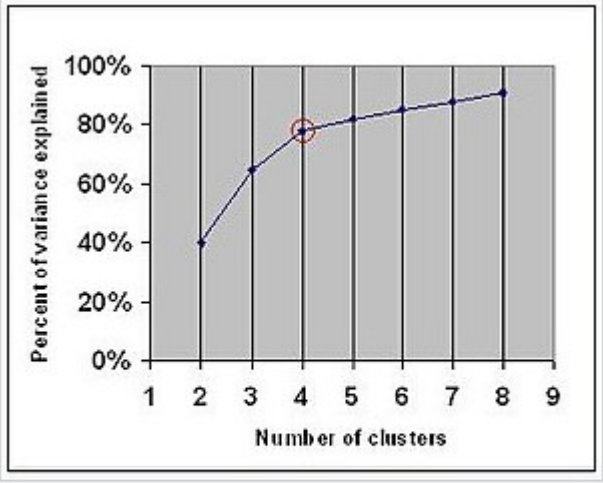
$\delta(Xi, Yj)$ is the intercluster distance i.e. the distance between Xi and Yj
$\Delta(Xk)$ is the intercluster distance of cluster Xk i.e.distance within the cluster Xk
However, the disadvantage of Dunn index is that with a higher number of clusters and more dimensions, the computation cost increases.

Silhouette Coefficient

Silhouette Coefficient tracks how every point in one cluster is close to every point in the other clusters in the range of -1 to +1.:
Higher Silhouette values (closer to +1) indicate that the sample points from two different clusters are far away.
0 indicates that the points are close to the decision boundary
and values closer to -1 suggests that the points have been incorrectly assigned to the cluster.

Elbow method

The elbow method is used to determine the number of clusters in a dataset by plotting the number of clusters on the x-axis against the percentage of variance explained on the y-axis. The point in x-axis where the curve suddenly bends (the elbow) is considered to suggest the optimal number of clusters.



Explained variance. The "elbow" is indicated by the red circle. The number of clusters chosen should therefore be 4.